

CONTENTS

1.ABSTRACT.....	(2)
2.INTRODUCTION.....	(2)
3. TOOLS AND TECHNOLOGIES.....	(3)
4. SYSTEM DESIGN AND METHODOLOGY.....	(4)
5. IMPLEMENTATION.....	(5)
6. RESULTS.....	(9)
7. DISCUSSION.....	(13)
8.CONCLUSION.....	(14)
9. REFERENCES.....	(14)
10. INSTALLATION AND EXECUTION.....	(14)

CHICKEN CROSS ROAD

[Project Report]

1. ABSTRACT

The Chicken Cross Road is a 2D arcade-style game developed using Java, Java Swing, and Java2D. The player controls a chicken and attempts to cross roads and rivers while avoiding moving vehicles and using floating logs.

The project implements player movement, collision detection, moving vehicles, river mechanics, score tracking, and dynamic level generation. The game follows an object-oriented and modular design where different classes handle specific game functionalities.

2. INTRODUCTION

The rapid growth of technology has made 2D arcade games popular for both entertainment and learning. These games help developers improve programming skills, game logic, and problem-solving abilities.

The Chicken Cross Road project was developed using Java, Java Swing, and Java2D to gain practical experience in game development and Object-Oriented Programming (OOP).

The main objective of this project is to develop an interactive game where the player safely crosses roads and rivers while avoiding moving vehicles and using floating logs. The game requires smooth player movement, collision detection, and dynamic level generation to provide an enjoyable gameplay experience.

The game includes roads, grass, rivers, moving vehicles, floating logs, trees, and a player-controlled chicken. It continuously updates the game environment, processes keyboard input, detects collisions, and generates new map sections. The modular design of the project makes it easy to maintain and expand with additional features in the future.

The main objectives are:

- To develop a 2D game using Java.
- To create moving vehicles with different speeds.
- To implement collision detection.
- To develop river crossing using floating logs.
- To implement a score system.
- To generate new map sections dynamically.
- To apply Object-Oriented Programming concepts.

3. TOOLS AND TECHNOLOGIES

Programming Language – Java

Java is the primary programming language used to develop the Chicken Cross Road game. It is an object-oriented programming language that provides platform independence, strong security, and excellent support for graphical application development. Java allows the project to be organized into multiple classes such as Player, Car, Log, GamePanel, and GameManager, making the code modular and easy to maintain.

Java Swing

Java Swing is used to build the graphical user interface (GUI) of the game. It provides components such as JFrame and JPanel, which are used to create the game window and display the game environment. Swing also handles keyboard input and screen updates during gameplay.

Java2D Graphics

Java2D is used for drawing the game objects, including the player, vehicles, logs, trees, roads, rivers, and other graphical elements. It provides methods such as fillRect(), fillOval(), and setColor() to create a simple but interactive 2D game environment.

Visual Studio Code

Visual Studio Code (VS Code) was used as the Integrated Development Environment (IDE) for writing, editing, and debugging the project. It provides Java extensions, syntax highlighting, debugging tools, and project management features that simplify the development process.

Java Development Kit (JDK)

The Java Development Kit (JDK) was required to compile and execute the project. It includes the Java compiler, runtime environment, and development tools necessary to build and run Java applications.

Git and GitHub

Git was used for version control, while GitHub can be used to store and manage the project repository. These tools help track project changes, maintain backup copies, and support future development.

Object-Oriented Programming (OOP)

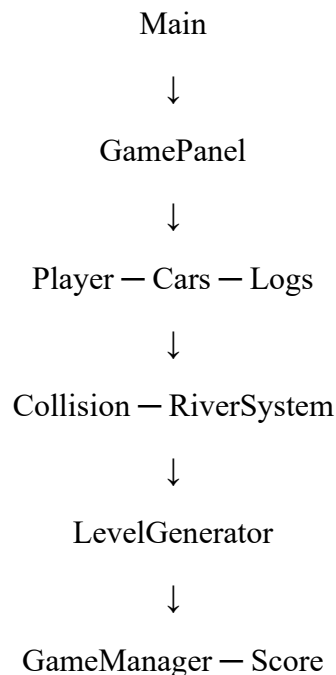
The project follows Object-Oriented Programming principles by dividing the game into multiple classes. Each class has a specific responsibility, improving readability, maintainability, and code reusability.

Examples include:

- Player – Controls player movement.
- Car – Controls vehicle movement.
- Log – Controls floating logs.
- Collision – Detects collisions.
- RiverSystem – Manages river logic.
- Score – Calculates and displays the score.
- LevelGenerator – Generates new map sections.
- GameManager – Controls the game state.

4. SYSTEM DESIGN AND METHODOLOGY

The project follows a modular architecture. The main classes communicate with "GamePanel", which controls the game loop.



The project was developed incrementally by implementing player movement, map design, vehicles, collision detection, river mechanics, scoring, level generation, and finally testing.

Methodology

The project was developed using an incremental development approach. Each week, a new feature was added and tested before moving to the next stage.

Week-wise Development

Week	Work Completed
Week 1	Project setup and Java environment configuration
Week 2	Player movement using keyboard controls
Week 3	Tile-based map, roads, rivers, and trees
Week 4	Vehicle system with multiple moving cars
Week 5	Collision detection and player reset
Week 6	River system and floating logs
Week 7	Score system and gameplay improvements
Week 8	Dynamic level generation
Week 9	Game management and optimization
Week 10	Testing, debugging, and final improvements

5. IMPLEMENTATION

Implementation Overview

The implementation phase focuses on converting the system design into a working application. The Chicken Cross Road game was developed using Java with an object-oriented approach, where each class performs a specific responsibility. This modular implementation makes the project easier to understand, maintain, and extend.

The game consists of several modules, including player control, vehicle movement, river mechanics, collision detection, score management, level generation, and game management. These modules work together to provide a smooth and interactive gameplay experience.

Main.java

The Main class is the entry point of the application. It creates the main game window and initializes the GamePanel, where all game activities take place.

Responsibilities:

- Start the application.
- Create the game window.
- Add the GamePanel.
- Start the game thread.

GamePanel.java

GamePanel is the central class of the project. It manages the game loop, updates all game objects, processes keyboard input, and redraws the screen continuously.

Main Functions:

- Initialize the game.
- Handle player movement.
- Update cars and logs.
- Detect collisions.
- Generate new map sections.
- Update the score.
- Draw all game objects.

Player.java

The Player class represents the chicken controlled by the user.

Responsibilities:

- Store the player's position.
- Handle movement.
- Return the player's collision boundary.
- Interact with vehicles, logs, and rivers.

Car.java

The Car class represents moving vehicles on the road.

Features:

- Stores vehicle position.
- Controls vehicle speed.
- Updates movement continuously.
- Returns collision boundaries.

Different cars move at different speeds, making the game more challenging.

Log.java

The Log class represents floating wooden logs on the river.

Responsibilities:

- Move continuously across the river.
- Carry the player safely while standing on the log.
- Reset position after moving outside the screen.

Collision.java

This class handles collision detection.

It checks whether:

- The player touches a vehicle.
- The player reaches dangerous areas.
- Collision occurs using rectangular boundaries.

When a collision occurs, the player is reset to the starting position.

RiverSystem.java

The RiverSystem class manages river-related game logic.

Functions:

- Detect whether the player is inside the river.
- Check if the player is standing on a log.
- Reset the player if they fall into the water.

Score.java

The Score class records the player's progress.

Responsibilities:

- Increase score during gameplay.
- Display the current score.
- Support future high-score implementation.

LevelGenerator.java

This class creates new map sections dynamically.

Instead of using a fixed map, new roads, rivers, and grass areas are generated automatically, increasing the replay value of the game.

GameManager.java

The GameManager controls the game state.

It is responsible for:

- Game over handling.
- Restarting the game.
- Managing important game events.

TileManager.java

The TileManager manages the tile-based map.

It draws:

- Grass
- Roads
- Rivers

This class creates the game environment used during gameplay.

KeyHandler.java

The KeyHandler class reads keyboard input from the user.

Supported controls:

- W / Up Arrow – Move Up
- S / Down Arrow – Move Down
- A / Left Arrow – Move Left
- D / Right Arrow – Move Right

Camera.java

The Camera class controls the visible area of the game.

Its purpose is to:

- Follow the player.
- Keep the player visible while the map changes.
- Improve the overall gameplay experience.

Summary of Implementation

The project was successfully implemented using a modular object-oriented architecture. Each class has a clearly defined responsibility, making the system easier to debug, maintain, and enhance. The combination of player movement, vehicles, floating logs, collision detection, scoring, and dynamic level generation creates an engaging and interactive arcade game.

6. Results

The Chicken Cross Road game was successfully developed and tested using Java, Java Swing, and Java2D. The final game includes all the planned features and runs smoothly without major errors. The player can move freely using the keyboard, avoid moving vehicles, cross rivers using floating logs, and earn points while progressing through dynamically generated levels.

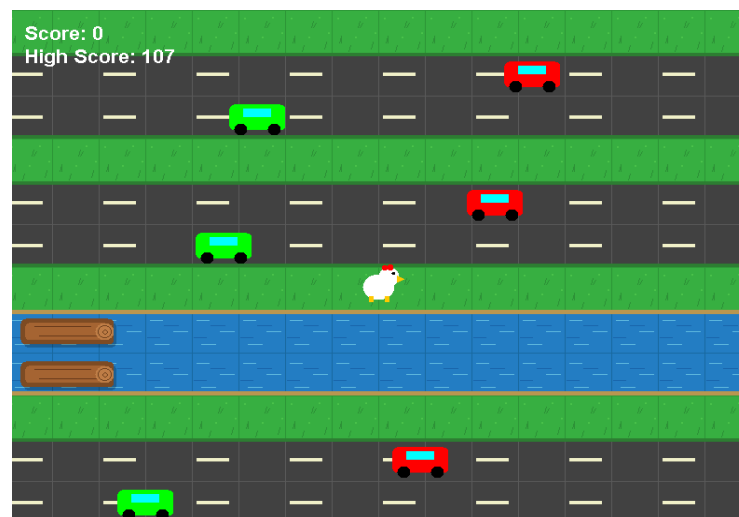
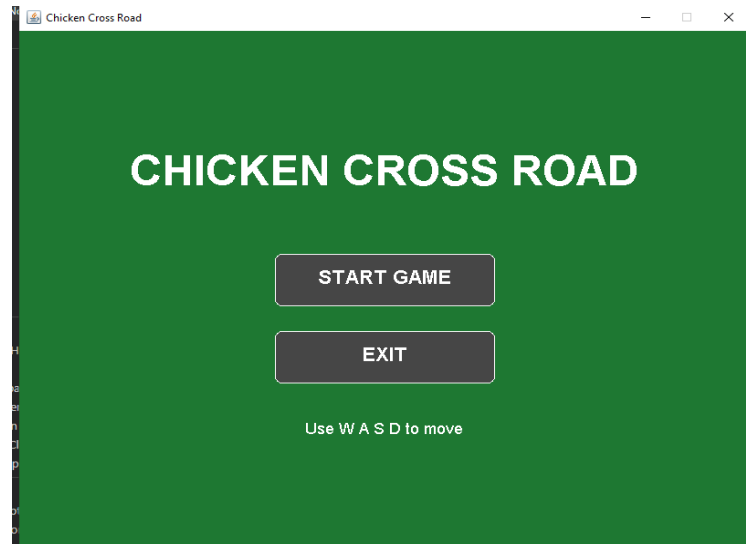
The completed game demonstrates the successful implementation of object-oriented programming concepts, collision detection, tile-based map rendering, and real-time game updates. The integration of different game components provides a smooth and interactive gaming experience.

Achieved Features -

The Chicken Cross Road game includes several interactive and dynamic features that provide an engaging gameplay experience. Each feature has been designed to improve the functionality and challenge of the game.

Start Screen

The game begins with a start screen that displays the game title and allows the player to start the game.



Player Movement

The player controls the chicken using the keyboard and can move in four directions (up, down, left, and right). The movement is smooth and grid-based, allowing precise navigation across the game map.

Moving Vehicles

The game contains multiple vehicles that move continuously across the roads at different speeds. The varying speed of each vehicle increases the difficulty and makes the gameplay more challenging.

Road and River Environment

The game world consists of different environments, including roads, grass, and rivers. Each environment provides unique gameplay mechanics that require different strategies from the player.

Floating Logs

Floating logs move across the river and help the player cross safely. The player must stand on a log while crossing the river to avoid falling into the water.

Collision Detection

A collision detection system is implemented to identify when the player comes into contact with moving vehicles or unsafe areas. If a collision occurs, the player is returned to the starting position.

Score Tracking

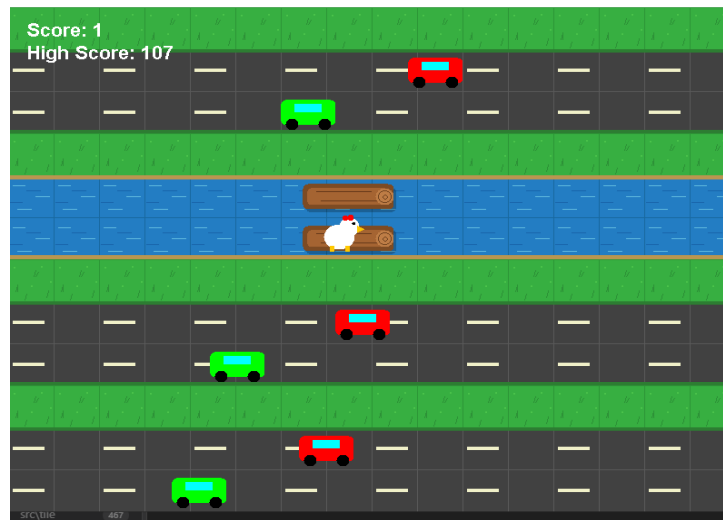
The game maintains a score based on the player's progress. The score increases as the player successfully advances through the game, encouraging better performance.

Dynamic Level Generation

The game uses a dynamic level generation system that continuously creates new sections of the map. This feature makes each gameplay session unique and increases replayability.

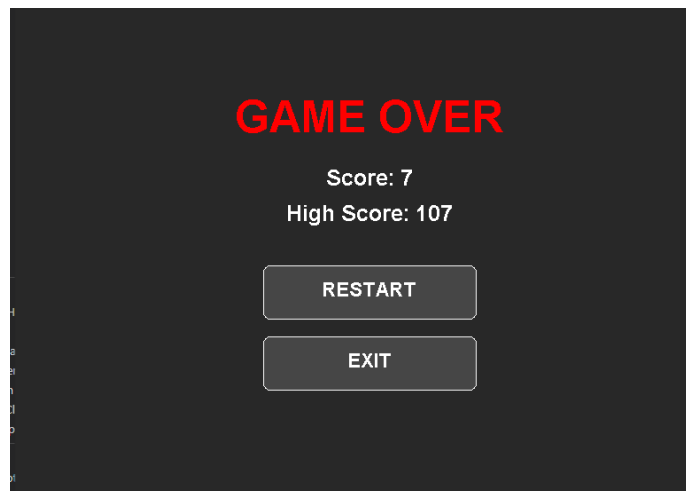
Tile-Based Map System

The game environment is built using a tile-based map system. Different tiles represent roads, rivers, and grass, making the game organized, scalable, and easy to maintain.



Game Over Screen

When the player collides with a vehicle or falls into the river, a Game Over screen is displayed. The player can choose to restart the game or exit.



Testing

The project was tested continuously during development to ensure that every feature worked correctly.

Test Cases

Test Case	Expected Result	Actual Result	Status
Player Movement	Player moves correctly	Working correctly	Passed
Vehicle Movement	Cars move continuously	Working correctly	Passed
Collision Detection	Player resets after collision	Working correctly	Passed
River Crossing	Player moves with logs	Working correctly	Passed
River Fall	Player resets without a log	Working correctly	Passed
Score System	Score increases correctly	Working correctly	Passed
Dynamic Level Generation	New map sections appear	Working correctly	Passed
Game Loop	Game updates continuously	Working correctly	Passed

7. Discussion

The project successfully demonstrates the basic concepts of 2D game development using Java. During development, several challenges were encountered, including collision detection, positioning of moving objects, river mechanics, and dynamic level generation. These challenges were solved by applying object-oriented programming techniques and dividing the project into multiple classes.

The modular architecture made it easier to debug individual components without affecting the rest of the game. Features such as moving vehicles, floating logs, score tracking, and dynamic level generation significantly improved the gameplay experience.

The project also provided valuable practical experience in Java programming, graphical user interface development, game loop implementation, and software design principles.

Challenges Faced

During the development process, several technical challenges were encountered:

- Maintaining proper spacing between moving vehicles.
- Adjusting the positions of floating logs inside the river.
- Implementing accurate collision detection.
- Synchronizing player movement with floating logs.

- Generating new map sections dynamically.
- Managing multiple game objects efficiently.
- Debugging runtime errors and improving code organization.

These challenges were solved through continuous testing, debugging, and code optimization.

8. Conclusion

The Chicken Cross Road project successfully demonstrates the practical use of Java, Java Swing, Java2D, and Object-Oriented Programming in developing a 2D arcade game.

The project provided practical experience in player control, collision detection, game loops, graphics, scoring, river mechanics, and dynamic level generation. The modular structure also makes the game easier to maintain and improve.

9. References

1. Oracle Java Documentation.
2. Java Swing and Java2D Documentation.
3. Git and GitHub Documentation.

10. Installation and Execution

The Chicken Cross Road project is available on GitHub. Users can download the source code, compile the project, and run the game by following the steps below.

Step 1: Clone the Repository

Clone the project using the following command:

```
git clone https://github.com/nowshentanisha/Chicken-Cross-Road.git
```

Alternatively, users can download the project as a ZIP file from the GitHub repository and extract it.

Step 2: Install Java

Install Java Development Kit (JDK 21) or the required Java version.

Step 3: Open the Project

Open the project folder in Visual Studio Code, IntelliJ IDEA, or any Java-supported IDE.

Step 4: Build the Project

Compile the Java source files or use the IDE's build feature.

Step 5: Run the Game

Run the Main.java file to start the game.

Step 6: Play the Game

- Click Start to begin the game.
- Use the keyboard to control the player.
- Avoid moving vehicles while crossing the road.
- Cross the river safely by standing on floating logs.
- Earn points as you progress through the game.
- If the player collides with a vehicle or falls into the river, the Game Over screen will appear.
- Select Restart to play again or Exit to close the game.

GitHub Repository

The complete source code of the project is available on GitHub.

Repository:

<https://github.com/nowshentanisha/Chicken-Cross-Road>